The compiling of Coreboot (Linux BIOS)

for Vorex86EX platform

Firstly, developer needs a workable Linux system for Coreboot compiling, the version Debian7.0 was suggested! And make sure few specific packages as gcc/g++, make, ncurses-dev, iasl and python were installed on the system already.

1, FETCHING COREBOOT

--Method 1--

you can download the Coreboot from out FTP server to make sure the version consistency as the same as we tested.

```
# wget ftp://vxex:gc301@ftp.dmp.com.tw/Coreboot/coreboot.tar
# tar xvf coreboot.tar
# cd coreboot
```

--Method 2--

- The installation of necessary packages

```
# apt-get install <package name>
```

- Installing the Coreboot package and compiling the Coreboot BIOS

```
# su
# apt-get install git-core
# git clone http://review.coreboot.org/p/coreboot
# cd coreboot
# git pull
```

2, COMPILING

- Download the revised Seabios with wget first,

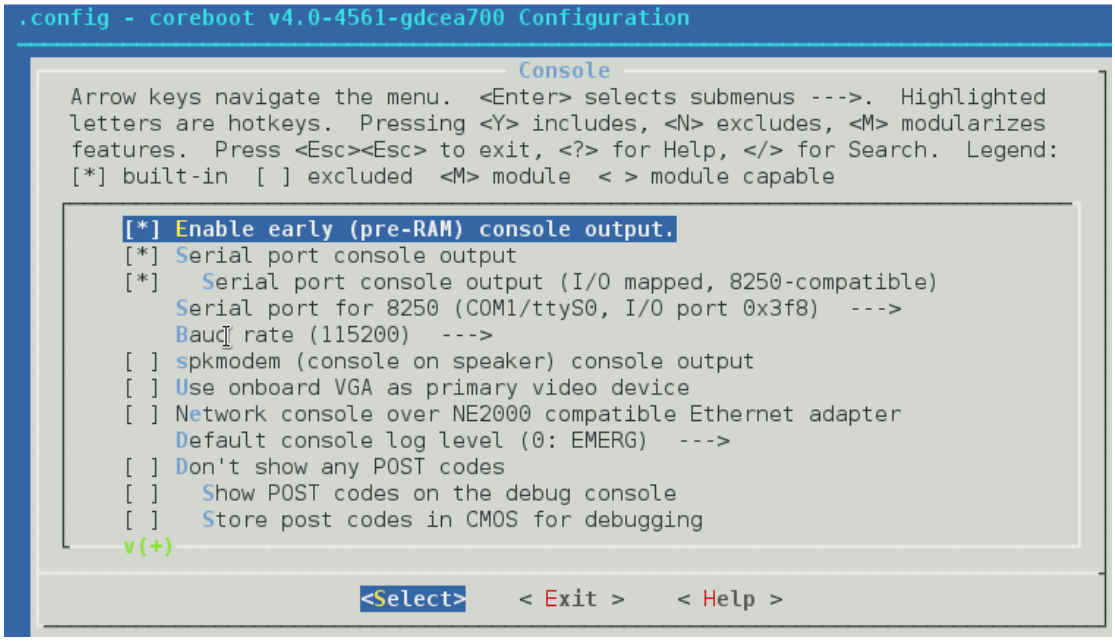
```
# wget
ftp://vxex:gc301@ftp.dmp.com.tw/BIOS&Cfg/Coreboot/Seabios_revised/bios.bin.elf
```

※ A known USB detecting issue solved by this revised BIOS!

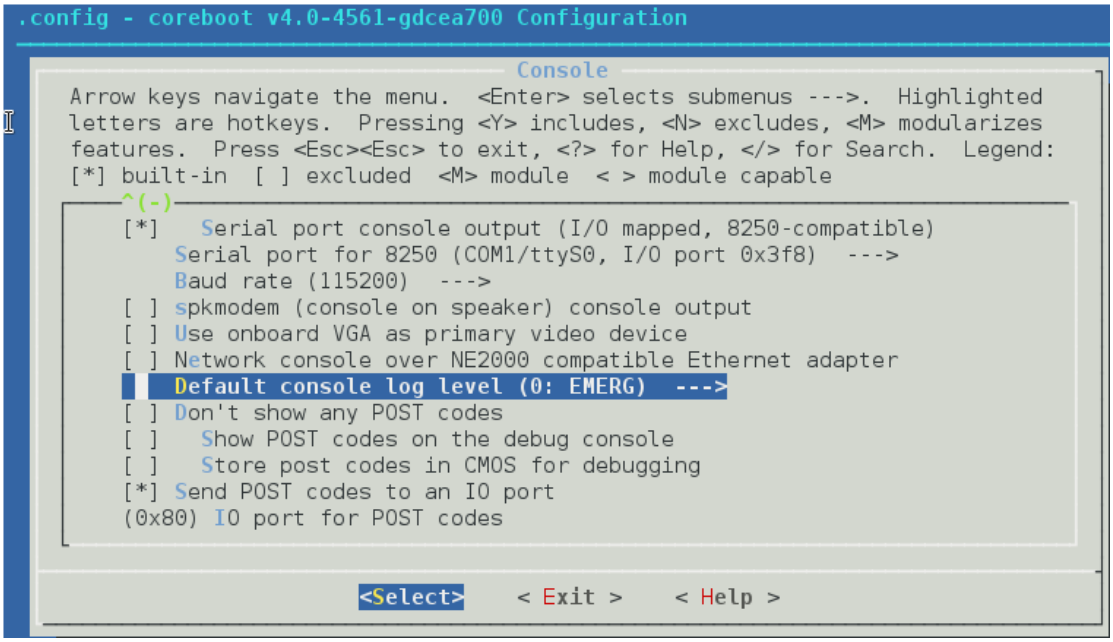
```
# make menuconfig
```

Suggested configuration for Coreboot:

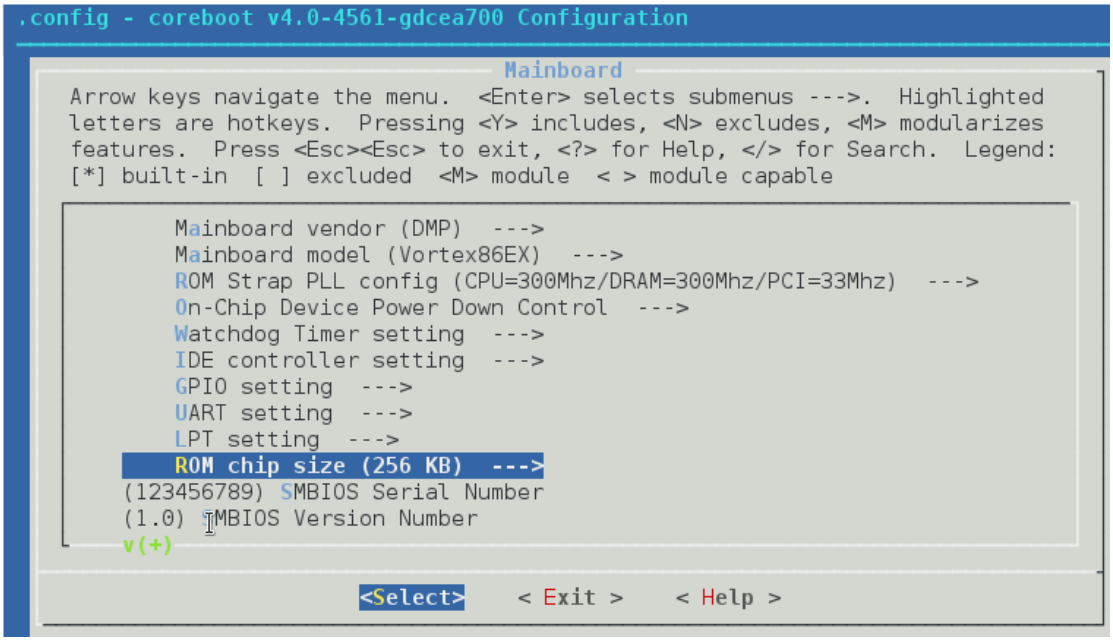
1, Check the "Enable early (pre-RAM) console output in option "Console"



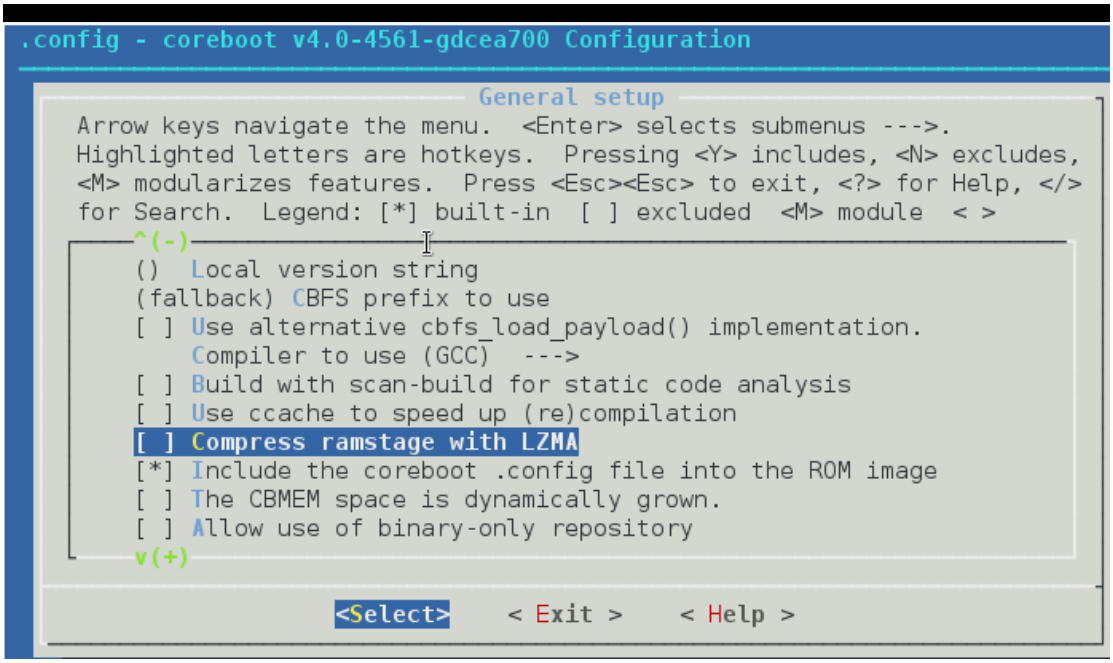
2, And click into the "Default console log level" to choose the (0: EMERG)



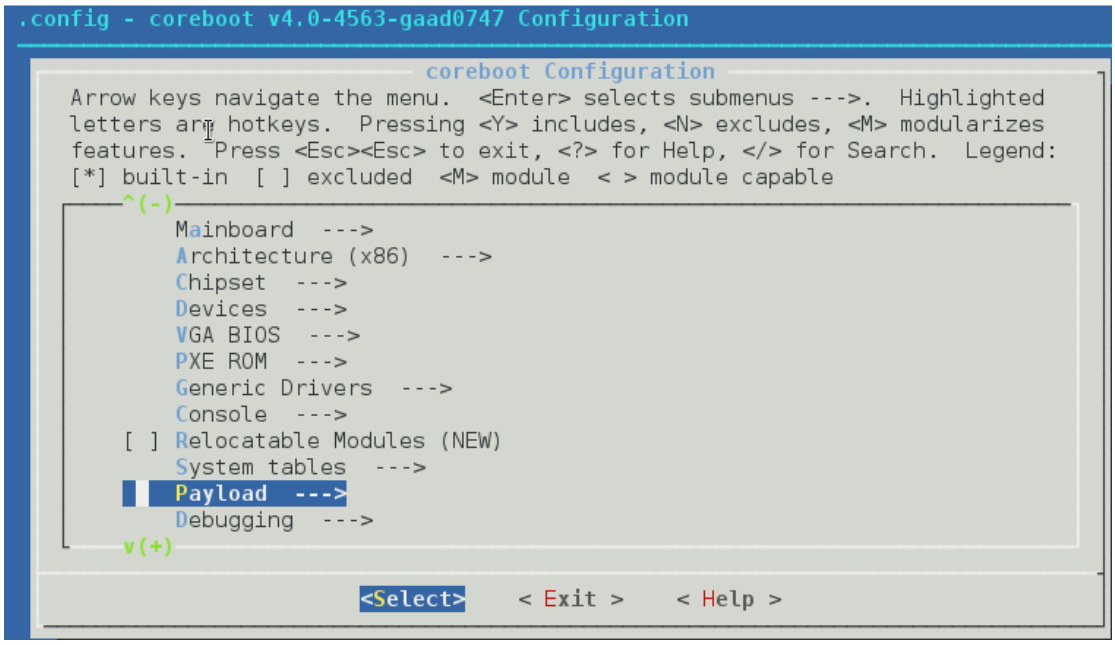
3, Suggest to build the Coreboot to conform to 256KB ROM chip size as normal.



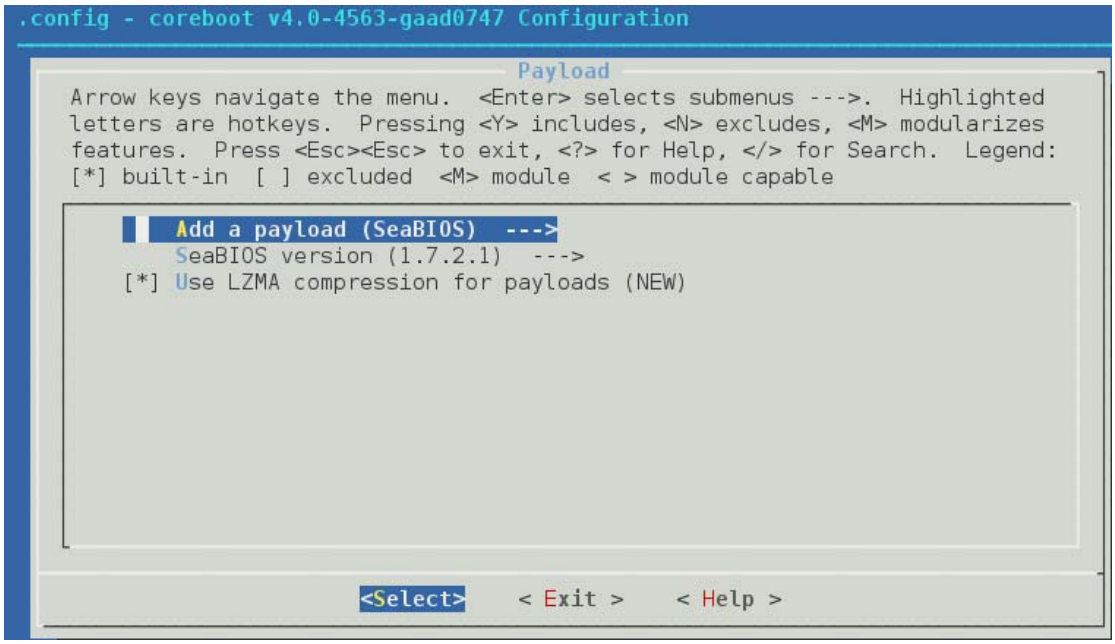
4, Uncheck the "Compress the ramstage with LZMA"!



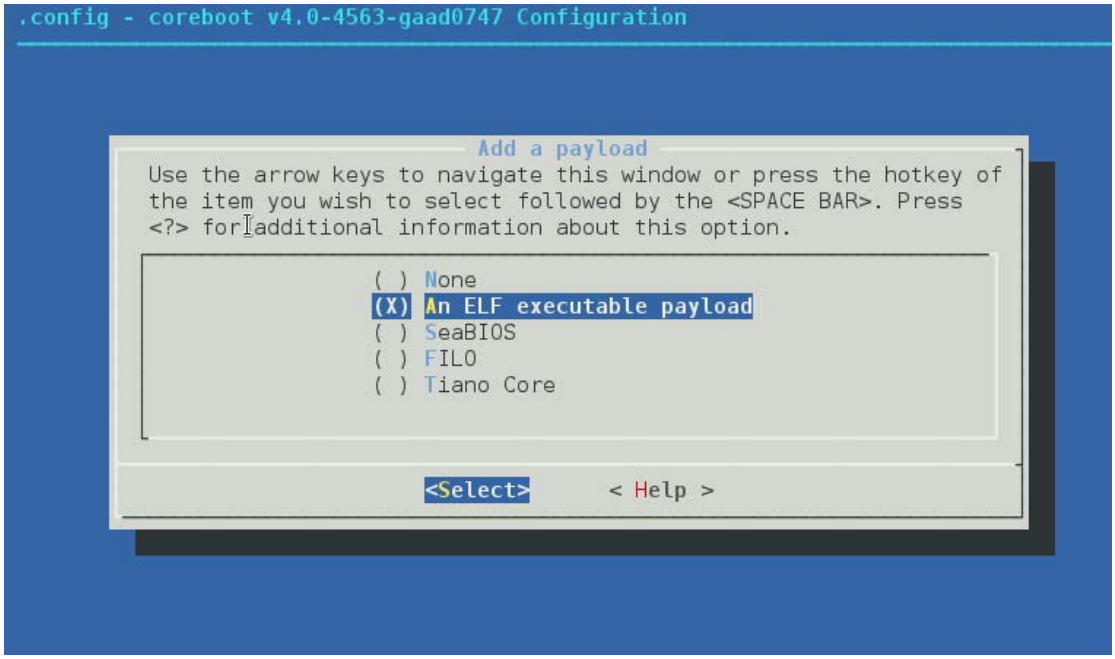
5, Insert the BIOS file "bios.bin.elf" that we got in previous.



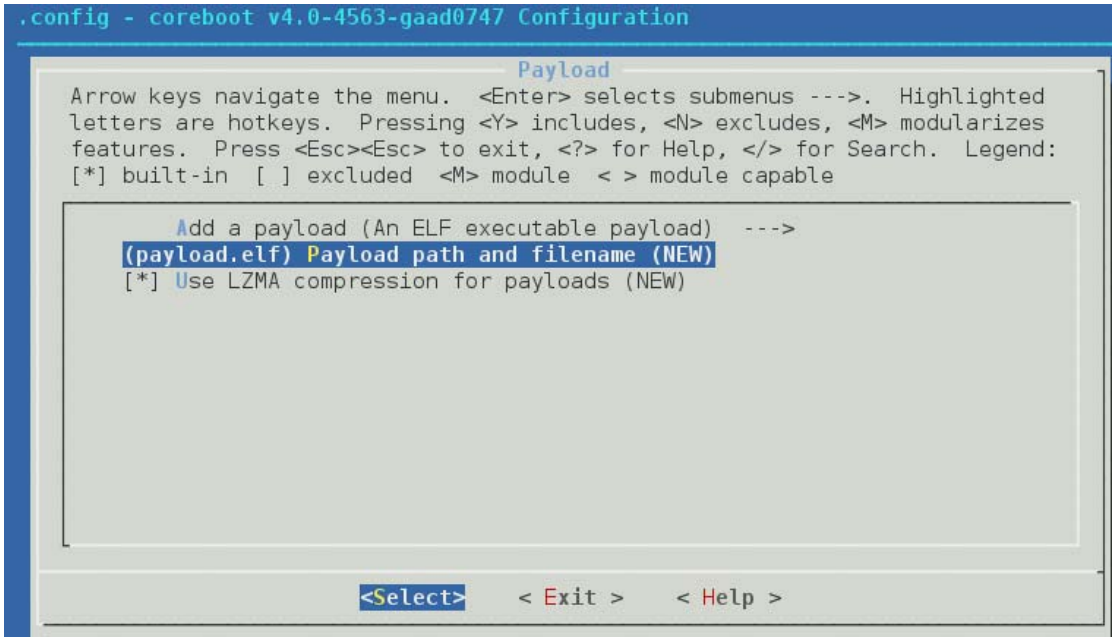
<Find the option of Payload>



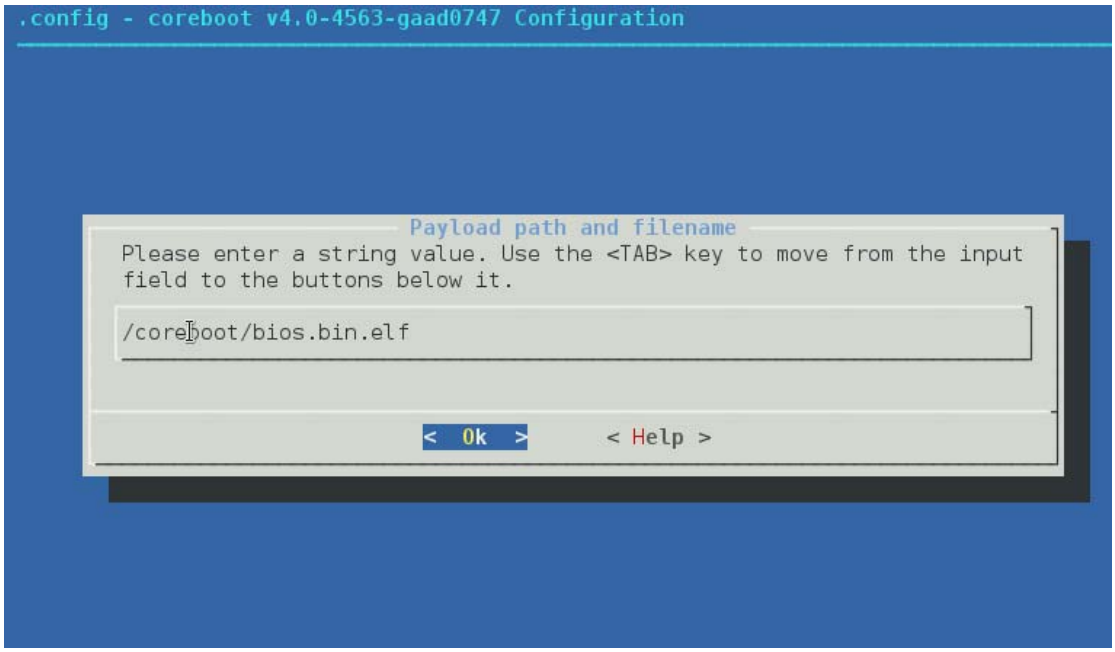
<Enter the "Add a payload (SeaBIOS)>



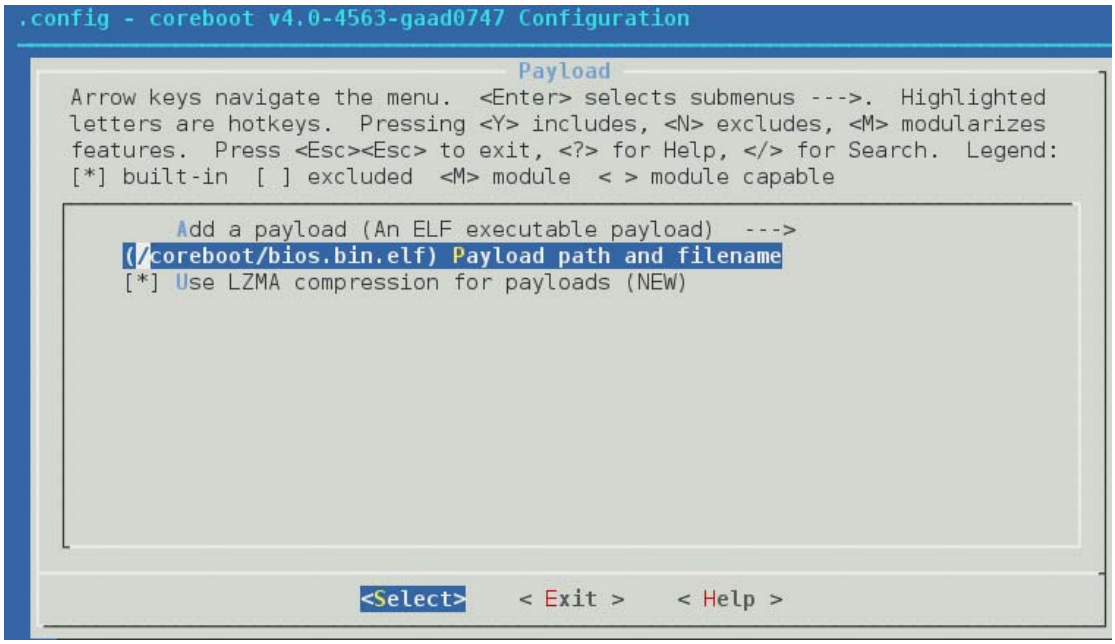
<Check "An ELF executable payload">



<Choose the "(payload.elf) Payload path and filename (NEW)">



<Typing the path of file bios.bin.elf located>



<SeaBIOS inserting was completed>

```
# make
```

And we would got the **coreboot.rom** produced in the folder of /coreboot/build/,

3, CROSSBAR FILE

- There would produced two files after done the configuration through

SocCfgTool tool (Ver.:20130430)

1, CROSSBAR-FILE: CROSSBAR-FILE : Crossbar data file, 240 bytes. (.bin file)

2, PLL-FILE : For CPU/DRAM speed setting.

(Don't need the PLL-FILE, cause of speed setting were included in the Coreboot tool already)

4, COMBINATION

- How to integrate CoreBoot BIOS with CROSSBAR-FILE?

Usage:

```
# python write-crossbar.py (ROM-FILE) (CROSSBAR-FILE)
```

Then system showing "Write CrossBar ok" when done!

```
root@debian:/coreboot_BIOS# python write-crossbar.py coreboot.rom parallel.bin
write-crossbar.py : Write CrossBar data into ROM file
ROM file : coreboot.rom, CrossBar file = parallel.bin
Write CrossBar ok
```

ROM-FILE : BIOS ROM file name, 256K (suggested) or 512K.

CROSSBAR-FILE : Crossbar data file, 240 bytes.

CROSSBAR-FILE data will be embedded into ROM-FILE.

5, UPDATING

--Method 1--



- Update the BIOS through the JTAG tool (5.7").

Copy the ROM-FILE to the dongle of JTAG tool, update the JTAG tool to version 1.52c before using, then get into the JTAG system and pressing key "D" to enter the DEBUG mode, executing the instruction as follow to complete the Coreboot updating.

```
:writerom <rom file>
```

--Method 2--



- Update the BIOS through the JTAG cable.

Connecting the JTAG cable on the parallel port of computer, to copy the JFLASH.exe in the root directory, then to execute "cmd" in the searching bar of Windows system, and to execute the updating as the instruction below.

```
C:\> jflash w <romfile>
```

Done!!